Milestone 3 Review Brief

Nexus Restaurant Management Platform: Sprint 3 delivery (AI document processing, costing, waste, sales and reporting), with a dedicated analysis of the document-parsing agent and an in-depth UI/UX audit

Prepared for	Clayton Linville (Product Owner). Section 5 is written to be forwarded to the PM.
Date	October 12, 2026
Delivery reviewed	`the_sweetman_standard` PR #1 "sprint 3" (merged Oct 12, commit `db21873`), synced to `sweetman_standard_clone` as commit `b37dcba`. The tree was verified identical to upstream.
Compared with	Milestone 2 (Aug 6 audit) and the original MVP (`nexus_restaurant_manager`).
Your input used	Your Oct 7 emails (Kanpachi / cashew-oil walkthrough, All-locations upload), the Oct 12 call summary, the Aug 10 and Sep 1 scope emails, and the SOW.
How it was reviewed	Eight specialist reviews in parallel: extraction, matching and confirm, model and prompt research, security, costing and units, test execution, and two UI/UX audits. All ran against a live local copy of the app. I re-checked the most serious findings myself. Then a live test of the extraction models on 17 of your own documents, 5 runs each (§5.9).

Executive summary

Sprint 3 is a **large and genuinely capable delivery**: 65,000 lines, 14 new data models, a complete upload → queue → review → confirm pipeline, unit-aware costing, menus, waste, sales import, variance reporting, an admin queue, and several Sprint 4 items early (Stripe billing, purchase recommendations). The engineering discipline is real. 429 unit tests and 740 API tests pass; lint, type checks and builds are clean; the handover documentation is excellent; and four Milestone 2 findings are properly fixed, including the cross-tenant analytics leak and unit-blind costing.

But the audit found problems that go to the heart of what Nexus sells: **trustworthy food-cost numbers from documents**, kept private to each restaurant group:

- **A stranger can end up as an owner of another customer's account** (SX-0, Critical, reproduced twice). The signed-in organization leaks between requests on a reused connection. A sign-up was placed, as owner, inside a different organization. The flaw was already in Milestone 2, and my August audit missed it.
- **The parsing agent can't be made consistent by settings, and nothing catches its mistakes.** No extracted line is checked against the document. The default model is Google's cheapest tier behind an alias that changes without notice. Every re-upload is a fresh random read. Word files feed the model hidden and deleted text. This fully explains the inconsistent and invented ingredients you saw (§5).
- **The live test on your documents confirms it, and shows the fix works.** As delivered, M3 put 90 wrong or invented values on the review screen in 85 reads, with no warning. The recommended pipeline (§5.7) let none through, found 99% of lines, and costs about a cent a document. On the handwritten recipe cards, sending the photos at full resolution took accuracy from 136 of 190 lines to 190 of 190 (§5.9).
- **Invoices can silently commit wrong prices.** "CS" (case) is read as "cup" (confirmed live on your Matt & Nat invoice): one test invoice moved oil from $8 to $202.88 per litre and a vinaigrette from $4.00 to $101.44. There are no arithmetic or invoice-total checks. A vendor code can bind a line to the wrong product. "Confirm all" commits parts nobody opened (§6, §7).
- **Your Oct 7 unit dead end is reproduced exactly, with its root cause:** a design that makes the natural fix ("1 cup = 218 g") useless and then locks the base unit (§7).

- **The food-cost variance report is wrong on real data:** it ignores purchases and physical counts, and the seeded demo reads −99% (§7).

Recommendation: don't treat Sprint 3 as done. Ask for the revisions in §10, in writing, before Sprint 4 work starts, starting with SX-0, invoice money safety and the extraction quick fixes. Most are small, well-defined changes; the full list is about 2–3 weeks of work. **On Fiverr:** the Sprint 3 order (#FO72A19A367C5, delivered Oct 2) was automatically marked complete on Oct 5, so Fiverr's revision button is no longer the route. Send the list to Viktor (cc Iuri) as Sprint 3 corrections, not new Sprint 4 scope, and consider holding your rating until it lands.

Severity	Count	Headline items
CRITICAL	3	SX-0 cross-tenant owner placement · DX-1 nothing checks extracted lines against the document · DOM-6/DX-3 "CS" read as "cup" on invoices
HIGH	14	Extraction: weak moving model, Word pollution, forced fields, pair mode, recall-only benchmark (DX-2/4/5/6/7) · Matching: wrong-product binds, invisible matches, SKU over name, unit-less lines, no totals, unseen parts confirmed, dollars as portions (DM-1–7) · Units dead end (DOM-1) · Variance report (DOM-2)
MEDIUM	22	Matching and units (DM-8–13, DOM-3/4/5/7/8/9), security (SX-1/2/3), extraction structure, truncation and chunking (DX-8–14); UI/UX items are counted in §9
LOW / INFO	22	Hardening and polish (DX-15–19, DM-14–17, DOM-10–14, SX-4–11)

1 · Scope, method and limits

- **Synced and verified** the Sprint 3 commit into your clone repo, preserving the team's authorship. Every file matches upstream except our own audit files.
- **Ran the whole product locally** (PostgreSQL 16, Redis, API, production web build), with separate databases so reviewers couldn't interfere with each other.
- **Eight parallel specialist reviews**, each told to separate *verified* (reproduced by running code) from *inferred* (read from code). Over 60 findings carry file:line evidence, and each verified one was reproduced with a small probe script.
- **I personally re-verified the two Critical findings** (SX-0 on the live app and in the database; CS→cup on the unit code), and cross-checked the Claude API facts against Anthropic's current documentation.
- **Ran every test suite**: backend unit, API end-to-end, lint, type checks, format, frontend build and the full browser suite (desktop and mobile).
- **Tested the extraction models live** with temporary keys you provided: 17 cases from your own files, 5 runs each, through M3 as delivered and through the recommended pipeline, scored line by line against answer keys I labelled by hand (§5.9).

Limits

- **The live test is a pilot, not the acceptance benchmark:** 17 cases, and the proposed prompt and checks were refined on the same documents they were scored on. The 30–50-document benchmark on unseen files (§5.8) is still needed. Your API keys were used only for this test and were never written to the repository.
- Staging (nexus-web.acw-test.space) was not logged into. Its real extractor setting is unknown: the AWS template sets no model name, so the default applies.
- The delivery is one squashed commit, so per-fix history couldn't be reviewed.

2 · The issues you reported: what's fixed and what isn't

From your Oct 7 emails, the Oct 12 call and earlier agreements. Each was checked against the code and, wherever possible, **reproduced** in the running app.

#	Your issue	Status	What we found
1	"All locations" selected: no uploads possible	**Partly**	The server now lets an owner upload recipes and menus organization-wide; invoices and sales still need a location, as you wanted. But the Documents page **defaults to Invoice**, so with All locations the drop zone is still disabled until the type is changed. Any user with location assignments (even to every location) still can't upload recipes or menus org-wide.
2	Kanpachi: cashew oil gets a unit from the recipe; no inline conversion; base unit locked	**Not fixed**	**Reproduced exactly.** The importer silently prices an unmeasured line ("for frying") per **each** at $0. In the recipe editor the error is only a toast, there's no base-unit control, and the unit picker can't even offer "cup" for an each-priced ingredient. The document-review screen gained an inline conversion and base-unit choice, but only when the file's own lines disagree.
3	Conversion added in Settings, saving still fails	**Partly**	**Depends on how the ingredient is priced.** For an ingredient priced by weight, a Settings conversion now works (verified: "1 cup = 0.218 kg" let an open recipe save). For the **each**-priced ingredient the importer creates from "cashew oil, for frying", which is your exact case, it still fails: the natural "1 cup = 218 g" can't reach "each" (§7). Settings is manager-only, and its default "Every ingredient" mis-costs everything else.

#	Your issue	Status	What we found
4	Pickers unusable with hundreds of ingredients	**Partly**	Tested with 318 ingredients. Ingredient pickers are grouped by category with a filter box, but picking still takes two controls, with no Enter-to-pick and no fuzzy matching. Sub-recipe, recipe and unit pickers have no search; the Ingredients list isn't paginated.
5	Same recipe, different ingredients; invented ones	**Not fixed**	The pipeline guarantees this can happen (§5): nothing checks lines against the document; weakest model tier on a moving alias; the model decides recipe structure; every re-upload is a new random sample.
6	"Add ingredient" on the recipe edit screen	**Fixed**	"+ New ingredient" on the new-recipe and edit screens inserts the ingredient straight into the line. Gaps: the name isn't pre-filled from what you searched, and the base unit **defaults to kg**, which is how a cup-measured oil gets locked out of cups.
7	Confirmation so edits aren't lost	**Not fixed**	**Verified in three places:** Esc or a click outside the document-review window, the Edit Recipe window, and the New Recipe wizard all throw away your work without asking. No unsaved-changes guard exists anywhere in the frontend.
8	Go to Documents automatically after upload	**Not fixed**	The upload window closes and you stay where you were, with a toast, a banner and a badge. A file that **failed** is never mentioned outside Documents, and after confirming there's no link to the recipe that was created.
9	Custom units; org-level unit settings; prompt to add a missing unit	**Not delivered**	The unit list is still hard-coded and the server refuses anything else. No organization unit settings. No "add this unit" prompt in the recipe editor.
10	Sales look like thermal receipts; need per-item detail	**Partly**	Data is stored per menu item, but the **Sales screen offers only daily totals and a raw line list**, with no per-dish view, no location column, and one dish per manual entry. "Variance by Dish" shows quantity but no revenue or food-cost %. A spreadsheet without a quantity column reads dollars as portions (§6 DM-7).
11	Auto-match exact or ~95% fuzzy; flag the rest (Aug 10)	**Partly**	Exact, plural, alias, vendor code and prep-word matching; **no fuzzy matching**. Weak matches are never shown to the reviewer, and prep-word stripping binds wrong products (§6 DM-1, DM-2).
12	Upload page silently did nothing (Sep 18)	**Partly**	The disabled drop zone now explains itself, but **dropping a file on it still does nothing**: no message (verified).

Score: 1 fixed, 6 partly, 5 not fixed or not delivered. The two that most affect daily use, the unit-conversion dead end (#2, with #3 fixed only for weight-priced ingredients) and inconsistent extraction (#5), are the least resolved.

3 · Milestone 2 follow-up

A correction first. The most serious finding in this audit, SX-0 below, comes from a line of code that was already in the Milestone 2 delivery. I read that file in August and didn't catch it. The M2 suites passed then, as the M3 suites pass now, because no test sends two organizations' requests over the same connection. It's reported here as new, with that history stated plainly.

ID	M2 finding	Status	Notes
M2-H1	Cross-tenant leak in cost-by-category	**Fixed**	Verified with a two-org probe across 10 analytics endpoints.
M2-H2	Costing ignored units	**Fixed**	Unit-aware costing with conversions; factors exact; nested costing verified numerically. Usability gaps remain (§7).
M2-H3	Mailer was a stub	**Partly**	Real Resend integration in code, but the AWS deploy template still ships `MAIL_PROVIDER=none`, so forgot-password sends nothing.
M2-M1	COUNT overwrote org stock	**Fixed**	Verified: org total = sum of locations.
M2-M2	Cross-org reference IDs accepted	**Fixed**	Verified: ~35 foreign-reference writes refused.

ID	M2 finding	Status	Notes
M2-M3	Isolation rested on one app-side layer	**Partly**	Coverage test added, but database RLS is still inert for app traffic, and SX-0 shows the single layer failing.
M2-M4	Location pricing unused in costing	**Not fixed**	Declared deliberate. Consequence verified: in a multi-location org, **invoice prices never reach recipe or menu costs** (§7 DOM-3).
M2-M5	Unassigned users see every location	**Not fixed**	Kept by design and documented.
M2-L1/L3/L4	Refresh via GET; plaintext invite tokens; public Swagger	**Not fixed**	Unchanged.
M2-L2/L7/L8	Throttling; pagination; security headers	**Partly**	Mostly addressed.
M2-L5/L6/L9	Batch cost; rounding; secrets in logs	**Fixed**	Batch cost scales; 4-dp prices; logs masked.

4 · What was delivered, and the test results

Measure	Milestone 2	Milestone 3
Change size	—	384 files, +65,429 / −8,068 lines (one squashed commit)
Database models / migrations	24 / 11	38 / 30
API operations (route handlers)	86	148
New capabilities	—	Document pipeline (Gemini / Claude / pair, BullMQ queue, review, confirm, multi-document split, mapping screens); unit conversions; menus; vendor catalogues; waste; sales import; variance report; admin queue; *and early Sprint 4 items*: Stripe billing, purchase recommendations and drafts
Handover docs	—	HANDOVER, ENVIRONMENT, PROVIDERS, DEPLOYMENT, AWS task definitions

Test runs (executed during this audit, isolated database)

Suite	Result
Backend unit tests (29 suites)	**429 / 429 pass**
API end-to-end (17 suites)	**740 pass, 2 skipped** (S3 suite needs a MinIO server), **0 fail** (209 s)
Lint, type check, format (backend and frontend)	**Clean**
Frontend production build	**OK**
Browser suite, Playwright (desktop + mobile)	**264 / 264 pass** (253 desktop + 11 mobile), 0 fail, 0 flaky; about 14 minutes
Document-module unit tests	185 / 185 pass
Costing and rules unit tests	93 / 93 pass

How to read this: the team's suites are extensive and green, which is a real strength. But they test what the team designed. None covers two organizations sharing a connection (SX-0), "CS" on an invoice (DOM-6), variance against physical counts (DOM-2), or extraction precision (DX-5). Green tests and serious defects coexist here.

5 · Document processing: why imports are inconsistent, and what to do

This is the section you asked for. It is written to stand on its own, so you can forward it to Viktor as the findings from your Oct 12 action item. Every claim about the code was checked against the M3 source; items marked **verified** were reproduced by running the code. **The models were then tested live** on 17 of your own documents, 5 runs each (§5.9); measured figures replace the estimates where marked. Other model facts come from Google's and Anthropic's own documentation, retrieved Oct 12, 2026.

5.1 · How it works today

The pipeline shape is sound: upload → stored file → background queue (3 attempts) → extraction → unit normalisation → **review** → **confirm** (re-validated on the server, applied in one transaction). Nothing is applied without a confirm. Sales and invoice spreadsheets (CSV/XLSX) are parsed in code, without a model, and well tested. **Recipe and menu workbooks do go to the model**, as text in 20,000-character chunks, as do PDFs, images and Word files, to whichever model the `DOCUMENT_EXTRACTOR` setting selects:

Mode	Model (default)	Settings in the code	Assessment
gemini (most likely what staging runs)	`gemini-flash-lite-latest`, which today resolves to `gemini-3.5-flash-lite` (measured)	temperature 0, no thinking setting, 16k output cap, prompt sent before the file	Google's cheapest tier, behind an alias Google swaps without notice. Google says temperature is **ignored** on current Flash models and advises against values below 1.0 on Gemini 3. Not suitable for accuracy-first extraction.
pair	draft `gemini-3.5-flash-lite`, check `gemini-3.8-flash`	thinking 'low' on both; the checker returns add/remove/replace corrections	The checker can **add** lines with no proof. A verified merge bug renames lines. If the check fails, the unchecked draft is used, and only a server log records it.
claude	`claude-opus-5-5`	effort forced to 'low' (Anthropic's default is 'medium'); JSON-schema output; 16k cap	Strongest model on offer, run at its weakest setting. The request shape is correct. A max_tokens cut-off isn't checked, so a long document fails as "unreadable".

Staging's real setting couldn't be checked from the repository. The AWS task definition sets no model name, so the default applies. Nothing records which model produced an extraction: the `Extraction` table stores only the payload and a confidence that is always empty for model results. **Ask the team to confirm staging's `DOCUMENT_EXTRACTOR` and `GEMINI_MODEL` values.**

5.2 · Why the same recipe gives different ingredients, ranked by likely impact

First, a fact that reframes the problem: no current hosted model is deterministic, and no setting makes it so. Google states that temperature is ignored on current Gemini Flash models and will be rejected by future ones. Claude models reject it outright, and Anthropic notes that temperature 0 "never guaranteed identical outputs". In one published test, 1,000 runs at temperature 0 produced 80 different answers. **Consistency therefore has to come from the pipeline, not the model:** every imported line must be provable against the document, and anything that isn't is flagged rather than imported.

#	Cause	Severity	Status
1	**Nothing checks an extracted line against the document.** No line carries the printed text it came from, its page, or a confidence. No code asks "is this word in the file?". An ingredient mentioned only in the method, a "typical" ingredient filled into an illegible line, or one from a neighbouring recipe looks exactly like a real line in review.	Critical	Not fixed

#	Cause	Severity	Status
2	**A weak model behind a moving alias.** Flash-Lite at its default (minimal) thinking level reads multi-column recipe pages and phone photos, the hardest case for the cheapest tier. The alias changes target on Google's schedule, so behaviour can change with no code change, and nothing records the model used.	High	Not fixed
3	**The model decides the recipe's structure.** Your Kanpachi sheet (tartare + taro cups + dressing) can validly come back as one recipe with three sub-recipes, one flat list, or three documents. Each shape gives a different ingredient list. The prompt also asks for "the quantity the recipe uses" for a component, which is often not printed. When the model leaves it out, code inserts "1 portion".	High	Not fixed
4	**Every re-upload is a fresh random sample.** The file isn't fingerprinted, so an identical file is never recognised and never gets the same answer twice.	Medium	Not fixed
5	**Word files feed the model text that isn't on the page** (*verified*): tracked deletions, hidden text, every text box twice, and field codes. An edited recipe therefore shows ingredients you can't see in Word.	High	Not fixed
6	**The schema forces guesses.** Invoice unit price, sales unit price and menu price are *required*, so when a document prints only a line total or "Market price", the model must invent a number. Portions are asked for without "if stated". There's no "this isn't a recipe" answer, so a wrong-type upload is forced into the schema.	High	Not fixed
7	**Pair mode adds noise** (if used): two random passes; the checker can add unproven lines; a *verified* bug renames an inserted line to the sub-recipe ("sea salt 1 tsp" became "Ponzu 1 tsp"); a failed check silently ships the unchecked draft.	High	Not fixed
8	**The benchmark can't see hallucinations** (*verified*). It measures recall only: a perfect answer plus three invented ingredients and an invented sub-recipe still scores 100%. It has six synthetic snippets and no real documents. The PM's Sep 1 email committed to "a pilot accuracy benchmark … using a representative sample that we can mutually approve"; that hasn't been delivered.	High	Not fixed

5.3 · Verified defects in the pipeline (reproduced by running the M3 code)

ID	Defect	What it does to your data	Fix
DX-3	**"CS" (case) is turned into "cup".** The unit cleaner strips the plural "s" from "cs", leaving "c", which is an alias for cup.	A Sysco-style invoice with U/M = CS reached review with every line in **cups** ("TOMATO ROMA 25 LB, 2 cup, $31.50"). For oil, milk or cream, stock and cost per litre are off by orders of magnitude. For other items it triggers a meaningless "add a cup → kg conversion" prompt, the same kind of error you hit on Oct 7.	Separate invoice unit table (CS→case, CT→count, BX→box, PK→pack). Remove the bare "c" alias outside recipes. Add a test.
DX-4	**Word files leak hidden and deleted text** into what the model reads.	"2 cups heavy cream1 cup whole milk" (a tracked change glued together), a hidden "4 oz truffle butter (old version)", and a garnish line twice.	Remove deleted, hidden, struck-through and fallback text before reading, or render to PDF.
DX-9	**"for frying" is thrown away** when the model also returns a unit.	"cashew oil, for frying" became **0 cup**. This is the extraction half of your Oct 7 Kanpachi problem: the new ingredient's unit is then inferred from that phantom "cup".	Keep the phrase as an amount note; a line with no amount gets no unit.
DX-7	**Pair-mode merge restores names by list position.**	When the checker inserted "sea salt 1 tsp", it became "Ponzu 1 tsp". The salt was lost and the sub-recipe counted twice: a silently wrong recipe cost.	Retire pair mode, or match by name and require evidence on every change.
DX-5	**The benchmark scores recall only.**	Invented lines score 100%, so a prompt or model change that increases hallucination looks like no change at all.	Score precision and a hallucination rate; real documents; 5 runs each.

5.4 · Compared with the MVP

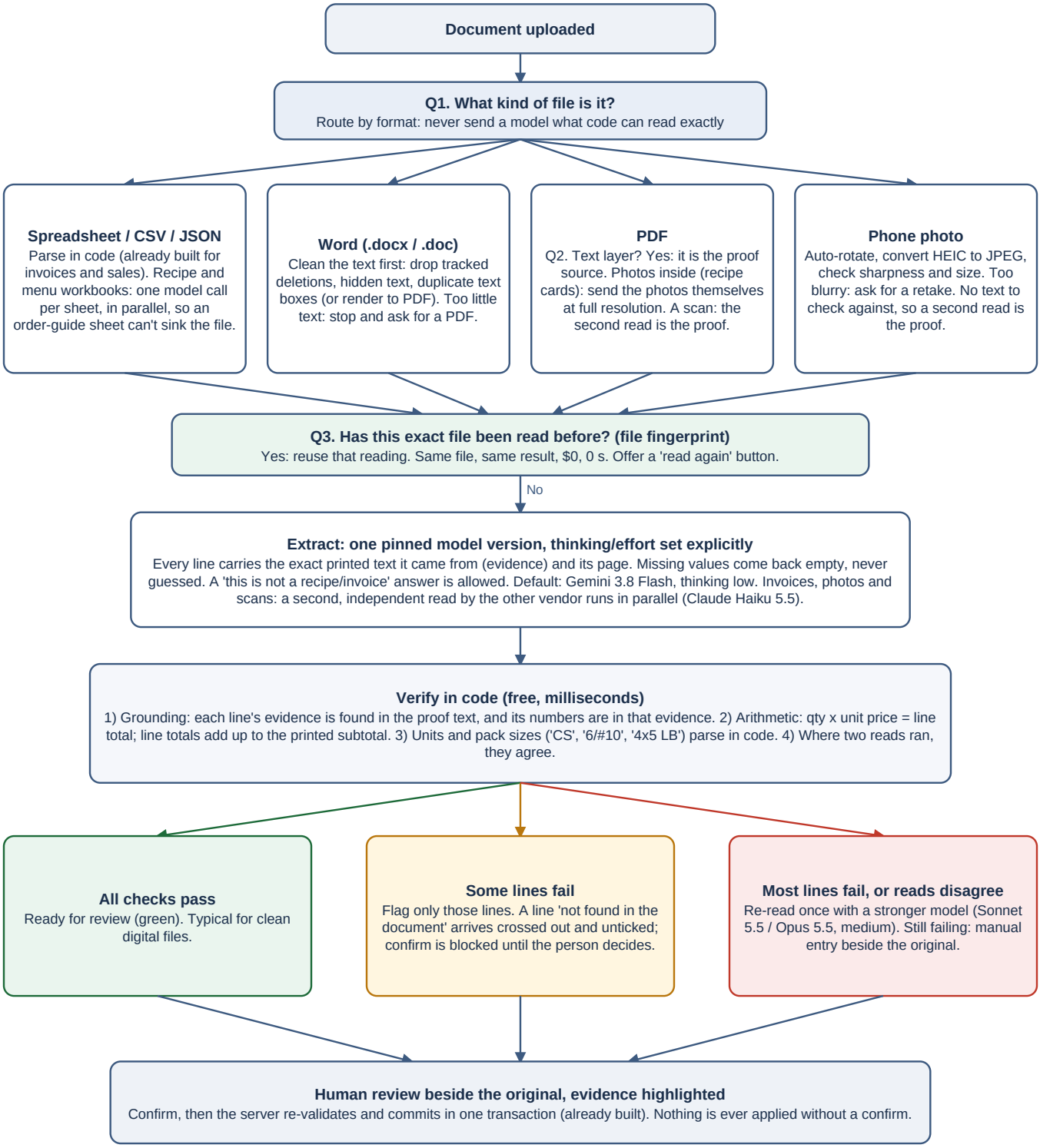
In one important way M3 is **safer** than the MVP. The MVP's prompts asked the model to invent: *"Estimate reasonable quantities from context (e.g., butter in a sauce = 2-4 tbsp)"*, *"use 1 as default"*, *"ALWAYS provide yield if not explicit"*, *"infer reasonable values for missing fields"*. M3 asks for names "exactly as written", allows an empty quantity, and says "never write 0". But M3 lost three good MVP ideas that should come back:

- **Inferred-field badges in review.** The MVP's schema required an `inferred_fields` list, and the review screen showed badges with approve/clear. M3 shows no signal at all; confidence is always empty.
- **A "this isn't that kind of document" outcome** (`extraction_failed`). M3 forces any upload into the selected type.
- **A pinned model ID** (`gemini-2.5-flash-preview-09-2025`, `claude-haiku-4-5-20251001`). M3 defaults to a moving alias.

Neither system checks extracted lines against the source document, and that is the gap that matters most for the problem you saw.

5.5 · Decision tree: accuracy first, then speed, then cost

Each step applies your priorities in order. **Accuracy** is protected by structure: never send a model what code can read exactly; require proof for every line; check that proof in code; let a person decide anything unproven. **Speed** comes from parallel reads rather than sequential ones, from reusing earlier readings of identical files, and from free code checks in place of second model passes. **Cost** is a minor factor at these volumes, as §5.7 shows.



Every check in the middle of the tree runs in code, takes milliseconds and costs nothing. The only paid steps are the model read and the second read for invoices, photos and scans (together about one to two cents a document, measured in §5.9).

5.6 · The options, with pros, cons and a verdict

Figures marked **measured** come from the live test (§5.9, average over 85 reads of your documents); the others are estimates from list prices of Oct 12, 2026. Gemini 3.8 Flash prices **double on Jan 1, 2027** ($0.75→$1.50 in, $3.75→$7.50 out per million tokens).

Option	Pros	Cons	Verdict
A. Keep today's setup (Flash-Lite alias)	Cheapest (measured $0.0033/doc); fastest (p50 1.6 s, p95 11 s).	Measured: 90 wrong or invented values reached review unflagged in 85 reads; 45% of reads clean; silent model swaps.	**Not acceptable**
B. Pin a stronger model only (Gemini 3.8 Flash, or Claude Opus/Sonnet 5.5)	Fewer misreads; stable behaviour; effort and model recorded.	Measured: Opus 5.5 in M3 still let 19 errors through ($0.039/doc). Switching the name to 3.8 Flash with no thinking level failed 6 of 85 reads (p95 103 s).	**Necessary, not enough**
C. Evidence per line + code grounding check + optional fields + a 'wrong type' answer	Invented lines can't pass silently, by construction. Works with any model. The biggest single gain.	Measured: 0 unflagged errors with all four models tried. About 2–4× today's time (p50 6 s, p95 40 s on 3.8 Flash). Needs proof text: free for digital files, a second read or OCR for photos.	**Do first**
D. Code validators (qty × price = total, lines add up to subtotal, pack-size parser, invoice unit table)	Free; catches misread digits that grounding misses; fixes CS→cup.	Engineering effort; vendor abbreviations need tuning (start with Sysco, US Foods, GFS).	**Do**
E. File fingerprint reuse	Identical file gives an identical result at $0 and 0 s; ends the visible flip-flopping.	Hides randomness rather than fixing it, so pair it with C. Needs a 'read again' button.	**Do** (quick)
F. Two parallel reads by one model; flag disagreements	Catches lines present in one run only; adds almost no latency when run in parallel.	2× model cost; errors the model makes consistently get through.	Superseded by G
G. Second read by the other vendor (Gemini + Claude Haiku 5.5) for invoices and for files with no text layer	Measured on handwritten cards and the invoice photo: 189 lines agreed and went green with 0 wrong; review dropped from 100% to 6% of lines.	Two vendors' keys; both can share a misread ("3 tsp" read as "3 tbsp" by both in a stress test). Haiku adds about $0.003 a document.	**Do for invoices and photos**
H. Draft-then-check chain (today's 'pair' mode)	A second look at the document.	Measured: 62 errors reached review unflagged; checker output ran away on 2 reads; 2.5× the cost and 2.8× the time of today's default.	**Retire**
I. OCR text layer for scans and photos (AWS Textract / Google Document AI, ~$1.50 per 1,000 pages)	Independent proof text for grounding; OCR confidences flag smudged regions.	Integration work; weak on handwriting. Never send OCR text alone: on scans, image input scored 87.5 vs 46.5 for text alone in one study.	Phase 3, if G is not enough
J. Claude citations	Quotes and page numbers guaranteed to point at the document.	Can't be combined with structured JSON output (API rejects it), so it needs two calls; text-layer PDFs only.	Not now
K. Batch / Flex APIs (50% off)	Half price.	Minutes to hours of delay.	Bulk back-fills only
L. Pre-built invoice parsers (Document AI Invoice $0.10/doc; Textract Expense)	Tuned for invoices; field confidences.	Costlier than an LLM read; don't know food-service pack conventions; no recipes or menus.	Optional benchmark contender

Option	Pros	Cons	Verdict
M. Send photos inside a PDF at full resolution (not the whole page)	Measured on the handwritten Diego Salazar cards: 190 of 190 lines right, vs 136 of 190 today. The largest gain found for Gemini.	Slightly more input tokens; needs a PDF image extractor (pdf.js / poppler).	**Do** (quick)
N. One model call per workbook sheet, in parallel	A non-recipe sheet can no longer make the model drop the whole workbook (measured: 5/5 vs 2/5 runs complete). Faster on big workbooks.	More calls (cheap ones).	**Do** (quick)

5.7 · Recommended target and phased plan

Target: C + D + E + M + N everywhere; the reading model pinned to `gemini-3.8-flash` with thinking set to *low*; G (a Claude Haiku 5.5 second read) for invoices and for photos and scans; H retired. **Measured on your documents (§5.9):** no wrong or invented value reached review unflagged, 99% of reads fully clean, 99.1% of lines found, about $0.010 per document ($0.004 for an invoice with the second read), p50 6 s and p95 40 s per upload. For a 5-location group uploading 40 invoices per location per week, that is about **$4 a month**, a small fraction of one subscription. Gemini 3.5 Flash-Lite in the same pipeline is a fallback at about two-thirds of the cost: still nothing unflagged, but more lines sent to review (87% clean).

Phase	Changes
Phase 1 — quick fixes (≈ 3–5 dev-days; ask for these as Sprint 3 revisions)	• Fix CS→cup and add an invoice unit table (DX-3) • Clean Word text and stop on too-little-text (DX-4, empty-text guard) • Keep "for frying" / "to taste" notes; a line with no amount gets no unit (DX-9) • Pin a model version; remove temperature; set thinking/effort explicitly (never change the model without it); refuse '-latest' in production; record model, prompt version and token usage on each extraction • Send photos inside PDFs at full resolution (M); one call per workbook sheet (N) • Make prices optional; add a 'not this type' answer; "portions if stated"; delete "the quantity the recipe uses"; never insert "1 portion" • Check Gemini finishReason and Claude stop_reason; say "document too long, split it" • Retire pair mode (or fix the merge bug and forbid unproven additions) • File fingerprint reuse with a 'read again' button
Phase 2 — proof per line (≈ 1–2 weeks)	• Evidence-first prompts and schemas (full rewrites in Appendix B): printed text per line, page, explicit empties; numbers parsed in code • Code grounding check plus new review flags: line not in document, number not in document, only mentioned in the method, possibly missed line • Invoice reconciliation: capture subtotal/tax/total; flag mismatches over 1% • Review screen shows the original beside the extracted lines, with evidence highlighted (there is currently no way to open the uploaded file from review) • The real benchmark (below), run before and after
Phase 3 — scans, invoices, structure (≈ 1–2 weeks)	• Parallel second read by the other vendor for invoices, photos and scans; agreement turns lines green, disagreement flags them (G) • Image rotation and quality gate; OCR proof text only if G is not enough • Build the sub-recipe tree in code from flat, sectioned lines (ends structure-driven variation) • Escalate a failing document once to a stronger model

5.8 · How to know it worked: the acceptance benchmark

Without measurement, every change above is guesswork. The team committed to a mutually approved benchmark; this is a concrete definition to hold them to:

- **Gold set:** 30–50 of *your* real documents, each with a hand-checked correct answer. Include the Kanpachi recipe, multi-recipe PDFs, the Bonds Brothers / Crawfords master sheets, the Diego Salazar PDFs, at least 5 phone photos, Sysco/US Foods invoices with case packs, a credit and a fee, a Word file with tracked changes, a thermal sales receipt and a POS export.
- **Run each document 5 times** per configuration.
- **Report:** hallucinated-line rate (lines not in the document); line recall; field accuracy for quantity, unit and price after normalisation; run-to-run agreement; p50/p95 time to review; cost per document.
- **Proposed acceptance bar:** 0 unflagged hallucinated lines reach review; field accuracy ≥ 97% on digital documents and ≥ 90% on photos, with every miss flagged; ≥ 98% line agreement across the 5 runs (100% for an identical file, via reuse); p95 time to review ≤ 60 s.
- Rerun it before any model, prompt or schema change. Each run costs a few dollars.
- §5.9 is a first, smaller run of this benchmark, on 17 cases.

5.9 · Live test, Oct 12: what your real documents showed

With the temporary keys you provided, I ran your own documents through the M3 code exactly as it ships, and through the pipeline this section recommends. **17 test cases, 5 runs each, 8 setups** (≈2,071 model calls, ≈$22 in total). The cases: six recipe Word files (Brown Butter, Tuna Tartare, Onion Tartlets, Caviar & Blini, Duck Pancakes as both .doc and .docx); the Collard Greens cookbook chapter; the B&B Top Chef event sheet; five sheets of the Bond Brothers master workbook (the staff sheet was left out); the three Diego Salazar recipe-testing PDFs, whose recipes are handwritten cards photographed into the form; the Matt & Nat invoice, as the PDF and as a phone photo; a distributor-style invoice I built with case packs, a catch weight, a total-only line, a fuel surcharge and a credit; and two wrong-type uploads (the invoice as a recipe, a recipe as an invoice). I hand-labelled the correct answer for each, including transcriptions of the handwritten cards. Every extracted line was scored as correct, wrong value, invented, merged, extra or missed.

"Recommended" means the evidence-first prompt (Appendix B, as revised by this test), with numbers parsed in code; the code checks of §5.5; photos inside PDFs sent at full resolution; one call per workbook sheet; a pinned model with thinking set to low. M3 rows run the delivered code unchanged, including its own post-processing (unit coding, sub-recipe nesting).

Setup (85 reads each)	Lines found	Wrong or invented, reaching review unflagged	Errors caught and flagged	Clean reads	Same answer all 5 runs	Time p50 / p95	Cost per doc
M3 today: gemini mode, default alias	88.4%	**90**	0	45%	7 / 17	1.6 / 11 s	$0.0033
M3 pair mode	91.8%	**62**	0	67%	7 / 17	4.4 / 26 s	$0.0082
M3 gemini mode, model switched to 3.8 Flash	96.3%	**35**	0	71%	10 / 17	15.0 / 103 s	$0.0190
M3 claude mode (Opus 5.5)	98.3%	**19**	0	84%	11 / 17	4.2 / 37 s	$0.0389
Recommended + Gemini 3.5 Flash-Lite	97.3%	**0**	34	87%	11 / 17	2.7 / 18 s	$0.0067
Recommended + Gemini 3.8 Flash	99.1%	**0**	1	99%	12 / 17	6.3 / 40 s	$0.0100
Recommended + Claude Haiku 5.5 *	93.9%	**0**	10	84%	12 / 17	8.1 / 63 s	$0.0026
Recommended + Claude Sonnet 5.5 *	83.6%	**0**	10	85%	13 / 17	6.2 / 64 s	$0.0319

Clean read: no wrong or invented value, nothing missed. Time: the whole extraction for one upload. Cost: list prices of Oct 12, 2026. * The Claude rows' lower recall comes almost entirely from the event sheet: Sonnet read only about 15 of its 65 items in every run, and Haiku between 15 and 65 (next page).

What the measurements show

- **M3 as delivered lets errors straight through to review.** The default setup put 90 wrong or invented values on the review screen across 85 reads, with no warning on any of them, and only 45% of reads were fully clean. The strongest M3 option (claude mode, Opus 5.5) still let 19 through, at 4× the cost of the recommended pipeline. **The pipeline matters more than the model.**
- **The recommended pipeline let none through** with any of the four models tried: every wrong value was caught and flagged by the code checks. With Gemini 3.8 Flash, 99% of reads were fully clean and 99.1% of lines were found, at about one cent per document.
- **Invoices:** M3's gemini modes got 5 of 50 invoice lines right. The model read "CS" correctly; M3's own unit code turned it into "cup" on every case line (DX-3). The required price field pushed a line total into the unit price (lemons at $2.40 each instead of $0.80) and a per-pound price onto a whole case (a $536.22 tenderloin costed at $12.89). The fuel surcharge was imported as an ingredient in 17 of 20 M3 runs, Opus included. Recommended: 50 of 50, fees kept apart.
- **Handwritten recipe cards** (the Diego Salazar PDFs): M3 today got 136 of 190 lines. On the Pavlova card it returned nothing or only the section headings in 4 of 5 runs, and in the fifth invented "½ cup diced fresh mango". It read "Fiocchi" as "Gnocchi" in 5 of 5 runs. Sending the photos embedded in the PDF at full resolution, instead of the whole page, raised Gemini 3.8 Flash from 160 to 190 of 190. Opus 5.5 reads the whole page almost as well (189 of 190), at four times the cost.
- **Word tables:** on the Duck Pancakes .docx, M3 today returned two heading lines and lost all 13 ingredients, in 5 of 5 runs.
- **Wrong-type uploads:** M3 today read the bakery invoice uploaded as a recipe into a "recipe" in 5 of 5 tries (brownies and tiramisu as ingredients), and the Onion Tartlets recipe uploaded as an invoice into an "invoice" with $0 prices in 2 of 5. Every other setup refused correctly.
- **Switching the model name alone is not safe.** With M3's code unchanged, Gemini 3.8 Flash spends up to 15,700 tokens thinking (M3 sets no thinking level), hits the 16k output cap, and failed 6 of 85 reads (the cookbook chapter in 5 of 5 runs) with p95 103 s. Pin the model and set its thinking level together.

- **The default alias resolves to** `gemini-3.5-flash-lite` **today** (from the model version Google returns). Pair mode's checker twice produced runaway output (over 50,000 characters) on the Pavlova card and the read failed.
- **Consistency:** M3 today gave the identical answer in all 5 runs for 7 of 17 cases; the recommended pipeline for 12 of 17. File-fingerprint reuse (option E) makes it 17 of 17 for re-uploads of the same file.

The test also corrected the proposal

The first version of the evidence-first prompt (as first written in Appendix B) failed in three ways. Each failure was fixed, and the whole configuration was re-run from scratch. Appendix B now holds the tested version.

- **It merged comma lists into one line.** "Granitée: 1 pint Hot Water, 1 pint Coconut Milk, 1 Tbsp Kosher Salt…" came back as one ingredient of 1 pint. Fixed with a one-ingredient-per-entry rule, plus a code check that flags any name containing a second amount.
- **It dropped a mixed workbook as "not a recipe"** because the first sheet is an order guide (2 of 5 runs, even with a workbook rule). Fixed in code: **one call per sheet**, run in parallel (5 of 5 runs found all 14 dishes, and it is faster).
- **Claude Sonnet classified the event sheet as a "menu" and returned nothing.** Narrowing the wrong-document rule (return nothing only when there is no ingredient list at all) kept Gemini correct, but Sonnet still read only one block of the sheet (about 15 of 65 items) in all 5 runs, and Haiku varied between 15 and 65. One more reason to keep Gemini as the reader and use Claude as the second reader, where it did well.
- **Two loose code checks** let a misread through on a PDF whose form text happened to contain the dish name. For PDFs and photos, the evidence must now match the document's text exactly, and a line whose evidence shows an amount it lacks is flagged.

Caveat: the checks were tightened after seeing these documents, then re-run on the same documents. That makes these results optimistic. The §5.8 benchmark, on 30–50 documents the pipeline has never seen, is still the acceptance test.

Where people still need to look

With the recommended pipeline, the share of correct lines a person still has to check was about 5% for Word files, 0% for workbooks, about a quarter for invoices (the phone photo, and one computed price), and **100% for the handwritten cards and the invoice photo**, because there is no text to check them against. An independent second read by the other vendor fixes most of that. With Claude Haiku 5.5 as the second reader, lines where both read the same thing go green: 189 lines green with 0 wrong, 11 left for review (6%). With Sonnet 5.5: 186 green, 0 wrong, 7% left for review. Agreement is not proof: in a stress test against an older, error-prone first read, 2 of 20 wrong lines went green because both models misread "3 tsp" as "3 tbsp". Keep it for documents with no text layer, and keep the 'unclear' legibility flag.

6 · After extraction: how a wrong line reaches your numbers

Extraction is half the story. A correctly read line can still be **matched to the wrong ingredient**, or committed with a wrong unit or price, and a model's mistake can pass straight through. These were reproduced against the running app with real CSV/JSON uploads, which exercise the real matching and confirm code with no model involved. **Every one of the seven High items below changed stock or prices with no warning at review.**

ID	Severity	What happens (reproduced)	Effect on your numbers
DM-1	**HIGH**	**Prep-word stripping binds a line to a different product.** "crushed red pepper" matched the bell-pepper *Red Pepper*; "crushed tomatoes" and "peeled tomatoes" matched fresh *Tomatoes*; "rice, cooked" matched raw *Rice*; "chicken, roasted" matched raw *Chicken*. No warning.	Canned or processed goods costed at fresh prices; cooked weights costed as raw (cooked rice ~2.5–3× too expensive).
DM-2	**HIGH**	**The reviewer never sees what a line matched.** Review shows only the raw text, quantity and unit, plus a list of names that matched *nothing*. There is no "matched to" column and no confidence. The agreed "exact or ~95% fuzzy, flag the rest" isn't implemented; there's no fuzzy matching at all.	Every weak match (SKU, alias, plural, prep) is committed unseen.
DM-3	**HIGH**	**A vendor item code outranks the product name.** A line "Heavy Cream 36%" carrying code 1001 (Butter's code) was received **as Butter**, and Butter's price was overwritten from $10.00 to $4.50/kg.	An OCR digit slip or a reused vendor code moves stock and price onto the wrong ingredient.
DM-4	**HIGH**	**An invoice line with no unit is taken as the base unit.** "Parmesan, 2, (no unit), $60" became +2 kg, and the price went from $25 to $60/kg. That 2.4× jump is under the 10× warning threshold.	"2" cases or wheels become 2 kg; stock and price both wrong; every recipe re-costed.
DM-5	**HIGH**	**No arithmetic or invoice-total check.** "Chicken, 2 kg, $4.50, total $90.00" passed check with no issue, and Chicken went from $4.72 to **$45.00/kg**. The printed invoice total isn't even captured.	One misread digit, or one invented or missed line, commits a wrong price into every recipe.
DM-6	**HIGH**	**"Confirm all from this file" applies parts nobody opened.** In a two-invoice file, part 2 was never reviewed (its own check had a warning) but was committed, and it created a new ingredient "Saffron threads" at **$900/kg**.	Model-fabricated lines in unreviewed parts reach stock and prices.
DM-7	**HIGH**	**A sales sheet without a quantity column reads dollars as portions.** "Pasta Marinara, Net Sales 150.00" became **150 portions at $1**.	Theoretical usage and food-cost variance off by orders of magnitude.
DM-12	**MED-HIGH**	**Settings conversions default to "Every ingredient".** One org-wide *cup → kg* rule made Butter, Tomatoes and Chicken in cups all cost at the same density, with no warning. **This is very likely what happened when you "fixed" the Oct 7 cashew-oil error in Settings.**	Every cup of anything costed at one density, silently.
DM-8	**MED-HIGH**	**No duplicate-file detection.** An invoice with no number, uploaded twice, was confirmed twice (stock doubled). An overlapping sales report double-counted a day.	Doubled stock and sales.
DM-13	**MEDIUM**	**03/04/2026 is always read as March 4**, with no warning and no organization setting for date order.	Invoices in the wrong period, sales on the wrong day.
DM-10	**MEDIUM**	**Creating ingredients during import is one all-or-nothing checkbox.** Every unmatched name becomes a new catalogue item at price $0. Its base unit comes from the recipe's text, with no choice when the lines agree.	A hallucinated name becomes a permanent catalogue row in one click (milder form of your issue #2).
DM-9	**MEDIUM**	**Lines left unmapped at confirm are never replayed** after you map them, and re-uploading to recover them double-counts the rest.	Lost or doubled lines.

What works well here (verified)

- Confirm is safe against double-clicks and races: 4 parallel confirms gave one success and three refusals; everything a confirm writes commits in one transaction.
- The server re-validates every confirmed payload. Recipes with unmapped names, unknown units, missing conversions or circular sub-recipes are blocked.
- Your three named near-misses ("butter, unsalted" vs Butter, "cashew oil" vs Cashew, "lime juice" vs Lime) are correctly flagged rather than wrongly matched.
- Your Kanpachi structure now works: the taro-cup sub-recipe is created and linked as a component, and conflicting units now get an inline conversion and a base-unit choice during review.
- No cross-tenant path was found in the background worker; uploaded files are stored per organization.

Fix order for this layer (accuracy first)

- **Show every match and make weak ones accountable** (DM-2): a "matched to" column with the match type. SKU, alias, plural and prep matches need a one-click accept. Stop stripping words that change the product (crushed, peeled, cooked, roasted, ground, shredded) (DM-1). Flag code-vs-name disagreement (DM-3).
- **Invoice number sanity** (DM-4, DM-5): a missing unit is an error for weighed or measured items; flag qty × price ≠ line total; capture the printed total and flag a mismatch; lower the price-jump warning from 10× to about 2×.
- **"Confirm all" only applies parts that were opened** (DM-6), and never creates ingredients for unreviewed parts.
- **Sales parser must never guess the quantity column** (DM-7). Add an organization date-order setting and warn on ambiguous dates (DM-13).
- **Duplicates** (DM-8, DM-9): file fingerprint warning; same-vendor/date/total warning for invoices without a number; same-day warning for sales; an idempotent "apply newly mapped lines".
- **Density rules must name an ingredient** (DM-12). Review existing org-wide cross-dimension rules now; staging may already have one.
- **Per-ingredient create list** with editable name, unit and category (DM-10). Import recipes as drafts.

7 · Costing, units and inventory

Nested costing is **numerically correct**. It was checked by hand: by portion and by yield, cycles refused, price changes cascading. Unit factors are exact, physical counts now stay per location (M2-M1 fixed), and stock can't go negative. The problems are in what flows *into* costing, and in two reports.

Your Oct 7 unit problem: the exact root cause (DOM-1, High, reproduced)

- **Step 1, import:** "cashew oil, for frying" has no measurable unit, so the importer creates Cashew Oil priced per **each**, at $0, without saying so. (If the model also echoed a unit, the line becomes "0 cup" instead.)
- **Step 2, sub-recipe:** 2 cups of an each-priced ingredient can't convert, so it's refused, correctly.
- **Step 3, your fix:** "1 cup = 218 g" is accepted with "Conversion saved and recipes recosted", but it can't help. The resolver allows **one** cross-type step (volume→mass), and the ingredient is a *count*. Only "1 each = 1 l" works, which no cook would think to enter.
- **Step 4, change the base unit to litres:** refused, because the imported line references it. Recording any conversion also **locks** the base unit, so the error's own advice makes things worse. The recipe editor's unit dropdown only lists units reachable from "each", so "cup" isn't even offered.
- **Fix, in order:** (1) on that error, open an inline dialog pre-filled with the ingredient and both units: "how many litres is one each?", or "price Cashew Oil per litre instead". (2) Allow relabelling a base unit when price and stock are zero, and stop counting conversions as references. (3) Allow two-step conversions. (4) Never default an unmeasured import line to "each"; ask in review. (5) Custom units, if wanted, need an organization unit table.

ID	Sev	Finding (reproduced)	Fix
DOM-6	CRITICAL	**"CS" (case) is read as "cup" on invoices.** One one-location test invoice (1 CS canola oil, $48) moved the shared oil price from $8/l to **$202.88/l**, and a vinaigrette from $4.00 to **$101.44**. Review showed only a warning.	Invoice unit table; remove the plural-stripping; price jumps over ~2× become errors.
DOM-2	HIGH	**The food-cost variance report is wrong on real data.** "Actual" counts only manually keyed "use" movements plus waste, ignoring purchases and physical counts. 5 kg truly used showed $0 actual, and the **seeded demo reads −99% variance**. A kitchen that counts weekly would never see over-portioning or theft.	Actual = opening count + purchases − closing count, per item and location; show the basis.
DOM-3	MEDIUM	**In a multi-location org, invoice prices never reach recipe or menu costs** (only the location's price moves; documented as deliberate). That undercuts the core promise that invoice ingestion keeps food cost current.	A product decision: per-location costing, or a shared-price policy (e.g. latest or weighted average).
DOM-4	MEDIUM	An org-wide density rule re-costs every other ingredient (flour came out 74% high).	Density rules must name an ingredient.
DOM-5	MEDIUM	Waste is refused when book stock is short, so the waste record is lost.	Record the waste; flag the stock gap.
DOM-7/8	MEDIUM	Purchase suggestions ignore orders already approved; after a kg→g unit change, purchasing reads 7 kg of history as 7 g.	Subtract on-order; convert history on rebase.
DOM-9	MEDIUM	A recipe's "food cost %" is its share of the dish price, compared to the dish target: a dish at 55.6% showed its recipes at 22.5/25/6% "on target".	Show recipe cost % only at dish level, or label it clearly.

8 · Security and tenant isolation

CRITICAL

SX-0 — A stranger can become an owner of another customer's account (tenant context leaks between requests)

Where: backend/src/tenancy/tenant.middleware.ts:35 (`tenantStorage.enterWith(ctx)`) · **Evidence:** reproduced by the security reviewer and independently by me

The middleware stores the signed-in organization with `enterWith` and never clears it. When a connection is reused, the next request inherits it: login, sign-up, password reset, invitation acceptance and the billing webhook all run "inside" the previous visitor's organization. AWS load balancers reuse back-end connections across different visitors, so in production the previous visitor can be any other customer.

Reproduced three ways. (1) The Bella Cucina owner's correct password was refused 6 times out of 6 on a connection that had just served Golden Fork, and accepted 6 out of 6 on fresh connections. (2) A new sign-up on such a connection created its organization empty, but placed the new user, **as owner**, plus a "Main" location inside The Golden Fork Group. That gives full access to a stranger's recipes, prices, invoices and team. (3) A repeat sign-up collided with that stray location. The same line was in the M2 code.

Fix: Wrap each request in `tenantStorage.run(ctx, next)`, and run context-less requests in an empty scope; never `enterWith`. Add a regression test that sends two organizations' authenticated and public requests over one kept-alive connection. Then turn on database row-level security for real (FORCE RLS, a non-owner database role, `app.current_org` per transaction), so one layer failing is no longer fatal. **Fix before any further work, and before any real customer data goes on staging.**

ID	Sev	Finding	Fix
SX-1	MEDIUM	The seed makes the platform administrator an ordinary *admin* inside a customer org, whose owner outranks it. With mail off (the shipped template), the owner can issue a reset link for the platform admin, take it over and reach `/api/admin` for every tenant. Only a deployment note prevents this.	Make platform admins untouchable from tenant team routes; give them their own internal org.
SX-2	MEDIUM	**Verified:** a 74 KB Word file that expands to 25 MB was accepted and fully parsed. A higher-ratio file within the 15 MB limit can exhaust the API's 1 GB memory and take the app down for every customer.	Check uncompressed sizes before parsing; parse in a separate worker process.
SX-3	MEDIUM	No audit trail for platform-admin or team actions: reset links, deactivations, rewritten extractions.	Append-only audit log, visible to the owner.
SX-5	LOW	**Verified:** a deactivated or demoted user keeps their old powers for up to 15 minutes (role read only from the access token).	Re-check active and role on writes.
SX-6	LOW	`docker-compose.deploy.yml` runs production without APP_ENV, which silently enables **stub billing** (free self-upgrade) and the stub extractor. The AWS template is safe.	Refuse stubs whenever NODE_ENV=production.
SX-7/8/9	LOW	Upload type checked by file extension only; some latent gaps in the isolation helper; rate-limit store is per-instance.	Magic-byte check; extend the operation lists.

Verified solid

- Billing: Stripe signatures checked; each webhook event applied once; out-of-order events can't roll state back; the read-only (402) restriction for lapsed subscriptions works as designed.
- Document and file access across organizations and locations is fully blocked; files are stored per organization and there's no download route to abuse.
- The role ladder ("the team is run from above") is enforced; AWS IAM policies are least-privilege.

9 · UI/UX audit

Two UX specialists drove the live app with an automated browser at desktop (1440 px), tablet (1024 px) and phone (390 px) sizes, and looked at every screenshot. One went deep on the data-entry workflows (documents, review, recipes, ingredients, units, mapping), using a test file built to mirror your Kanpachi recipe. The other covered the rest of the app, mobile and accessibility. Severity: **Critical** = causes wrong data or blocks the task; **High** = major friction or a likely error.

Overall: the visual design is clean, modern and consistent, and the foundations are right: nothing is saved without review; the server checks the review as you type; a "Units across this file" resolver settles each unit once per file; names you map are remembered; deletes explain their consequences. But the touch points you care most about are **not yet painless**. Review is a 3,000-pixel pop-up that never shows the source document. The quickest way past a unit error writes wrong numbers. Work is lost with one stray click. And a cook still can't get past a unit conflict without a manager, a second browser tab and about 26 steps.

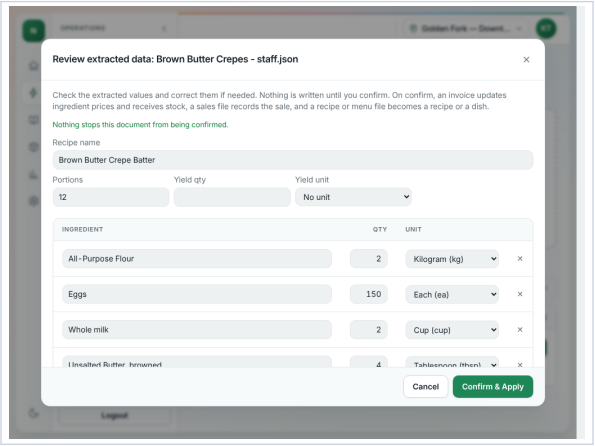
9.1 · What the core tasks actually take

Task (verified live)	Effort today	Main friction
Upload your Kanpachi recipe → review → map → confirm	~15 interactions, ~3,000 px of scrolling inside a pop-up, 2 page switches, **0 views of the source**	Fixes sit ~2,000 px below the red line; the source file can't be opened; no link to the created recipe afterwards; a failed Word file was never mentioned outside Documents.
Create a recipe using ½ cup of a new oil	~26 interactions, a second browser tab, and the **manager** role	"+ New ingredient" defaults to kg, so cups are hidden; the base unit is locked; conversions live in Settings. **A staff cook can't finish at all.**
Upload with "All locations" from Documents	2 clicks + 1 type change	The page opens on Invoice (not allowed org-wide), so the drop zone starts disabled; a drop on it does nothing.
Leave a half-edited recipe or review	1 stray click	**Everything is discarded, no warning.**

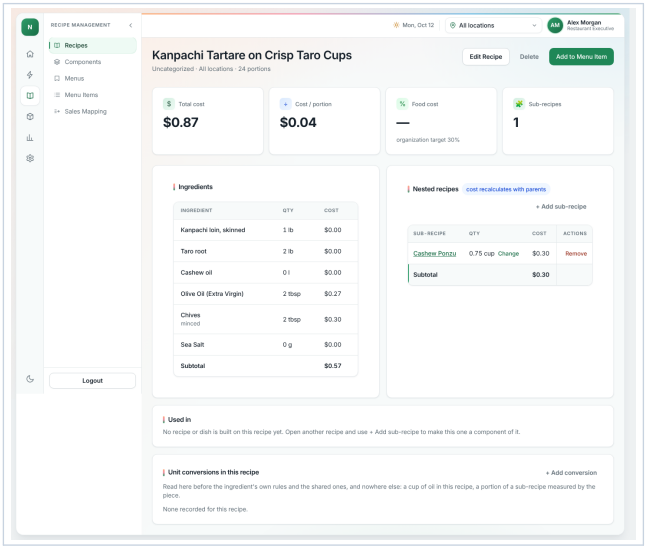
9.2 · Critical and High findings: core workflows

ID	Sev	Finding	Fix
UX-C1	CRITICAL	**The fastest way past a unit error writes wrong numbers.** In review, changing "2 cup" flour to "kg" without touching the quantity clears the error and enables Confirm: 2 kg of flour. "150 g" of eggs became 150 eggs. The error text itself suggests "choose a unit of the same kind", and for staff it's the only way through.	When a line's unit changes kind, open an inline converter ("2 cup → ? kg"); keep an "as written" chip; show a cost per line with outlier flags.
UX-C2	CRITICAL	**Edits are lost without warning** (your #7): Esc or a click outside the review, Edit Recipe or New Recipe window discards everything.	"Discard changes?" guard; no close-on-click-outside for forms; autosave review drafts.
UX-C3	HIGH	**The recipe editor has no way through a unit conflict** (your #2). Incompatible units are hidden; there's no inline conversion; the base unit can't be changed once used.	Inline unit bridge in the line (wireframe W1).
UX-C4	HIGH	**Review hides the evidence.** A 2,930 px pop-up in a 621 px window; no source view, no confidence, fixes far from errors; Confirm disabled with no reason given.	Full-page review workspace with the source beside the lines (W2).
UX-C5	HIGH	**One checkbox creates every unknown name as an ingredient**, unseen, at $0, with prep words in the name ("Kanpachi loin, skinned").	Preview table with editable name, unit, category, price and "did you mean" (W3).

ID	Sev	Finding	Fix
UX-C6	HIGH	**Unpriced ingredients silently under-state cost.** The imported Kanpachi tartare shows **$0.04 per portion** with no caveat (figure below).	"Unpriced" badges; "≥ $0.87 · 3 unpriced" on the totals; an "incomplete cost" filter.
UX-C7	HIGH	**Staff are blocked on unit conflicts** and shown input boxes that save nothing.	Let staff propose a conversion, or "send to manager".
UX-C8	HIGH	**Phones and tablets:** the review table is cut off; the Documents "Review" button is off-screen; the remove-line target is 12×20 px.	Card layouts below 768 px; 44 px touch targets.



UX-C1: after swapping units, "Nothing stops this document from being confirmed": 2 kg flour, 150 eggs.



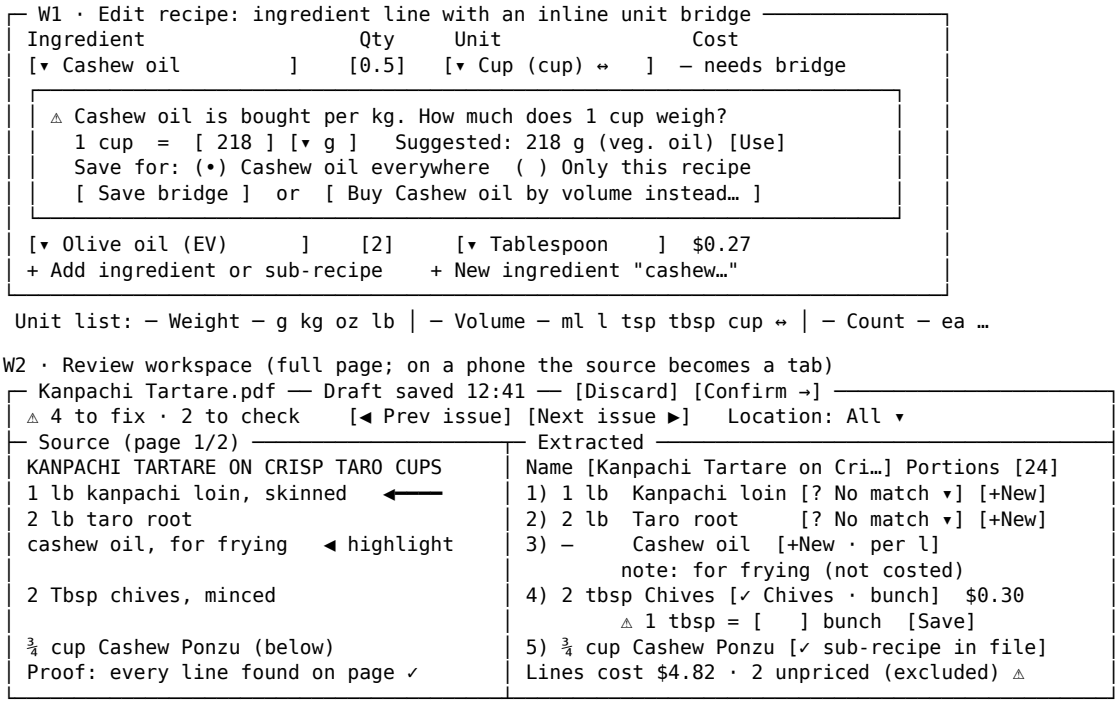
UX-C6: the imported Kanpachi recipe reads $0.04 per portion; cashew oil "0 l", with its "for frying" note lost.

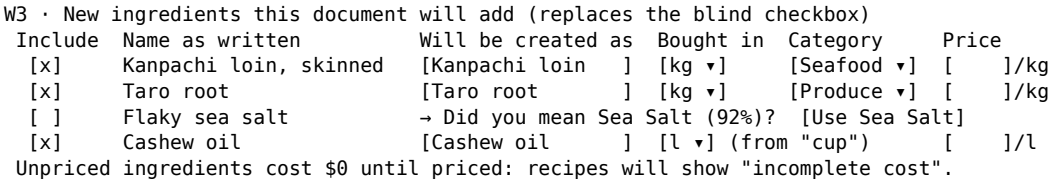
Medium items (core workflows)

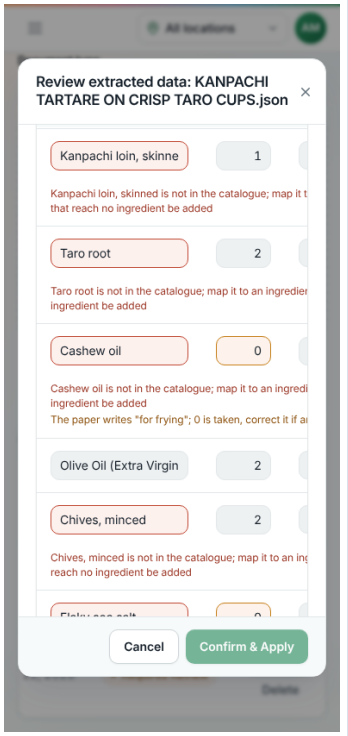
- No next step after upload or confirm (your #8): use a progress tray that opens review when a file is ready, plus "Open recipe →" after confirming. That's better than a hard redirect.
- Documents defaults to Invoice under All locations; put the location picker inside the drop zone, and explain a drop on a disabled zone (your #1/#12).
- Pickers at scale (your #4): one searchable combobox everywhere, with fuzzy match, Enter to pick, and "+ Create …" as its last option.
- The conversion dialog is abstract ("factor") and defaults to an organization-wide rule. Use a sentence builder instead ("1 cup of sesame oil weighs 218 g"), and require an ingredient for weight↔volume rules.
- Sub-recipes can't be added in the New Recipe wizard; "for frying" / "to taste" notes are lost on import; unmeasured lines are treated inconsistently (warning in one place, error in another).
- Errors are 4.5-second toasts far from the field; they should be inline and stay until fixed.

9.3 · Recommended redesigns

Three redesigns would remove most of the friction above. Each builds on patterns the app already has: the "Units across this file" resolver, searchable pickers, and server-side checking.







UX-C8 today on a phone: the review table is cut off at the right edge; units and the remove control are off-screen.

9.4 · The rest of the app, mobile and accessibility

Verdict: on a desktop the app is polished and consistent: one component library, careful copy, visible keyboard focus, reduced-motion support, a working dark mode, and an automated accessibility scan (axe-core) clean on 24 of 25 screens. **On a phone it is "phone-accessible", not "phone-ready"**, and the SOW requires mobile support for the core kitchen tasks.

SOW mobile item (390 px phone)	Verdict	Why
Login	**Pass** (with defects)	Works; iPhone Safari zooms on every field (14 px inputs); no show-password toggle.
Location selection	**Partial**	Two taps, but every Golden Fork location shows as "Golden Fork …"; the choice isn't echoed on the page.
Invoice upload	**Partial**	Disabled under All locations, with no location picker in the upload window; the review screen is cut off on phones (§9.2).
Recipe / menu upload	**Partial**	Same switcher friction; review layout as above.
Waste tracking	**Partial**	8 taps *if you search first*. Tapping an ingredient in the 318-item list opens the form **14,600 px further down**, so the tap seems to do nothing. Offline, "Log Waste" just stays disabled; closing the page loses the entry.
Inventory review	**Fail** (borderline)	On hand, Status and Adjust are off-screen to the right; counting is one pop-up per item, so **a 100-item count takes about 600 taps**.
Sales upload	**Partial**	The upload window works but is disabled under All locations; the data starts 1,787 px down the page; no per-dish view (#10).
Basic report review	**Partial**	Food Cost Analysis stacks acceptably, but variance columns are clipped and the figures can mislead.

ID	Sev	Finding (verified)	Fix
UX-A5	HIGH	**Food Cost Analysis presents an implausible result as good news:** actual cost at 0.8% of sales and a −96% variance coloured **green**, with no "not enough data recorded" warning. This is the UI face of DOM-2 (§7).	Data-coverage indicator; neutral "not enough data" state; show food-cost %.
UX-A23	HIGH	**A failed page load looks like real data:** a failed inventory request shows "0 items, $0.00"; a failed dashboard shows loading placeholders forever. No retry.	One shared error-and-retry state on every list and dashboard.
UX-A3	HIGH	**"All locations" silently disables actions** on Waste, Inventory, Sales, Production, Purchasing and uploads. Owners and every new organization start there; the reason is only in small print or a hover tooltip, which touch screens can't show.	Pick the location inside the action; default single-location users to their location; show the active location as a chip.
UX-A4	HIGH	**Your #10 isn't solved in the UI:** no "by menu item" sales view; manual entry is one dish per pop-up.	Product-mix tab with location breakdown and export; grid entry.
UX-A1/A2	HIGH	**Desktop tables on phones** hide key columns and actions; **no count sheet**.	Card layouts below 768 px; batch count sheet by storage area.
UX-A6/A20	HIGH	**Kitchen-hostile phone inputs:** 14 px fields force zoom; no decimal keypad; the waste form opens far off-screen.	16 px inputs, decimal keypad, form as a bottom sheet.
UX-A8/A9	MEDIUM	**Navigation and roles:** an icon-only two-level rail costs 2 clicks per destination. Role names mix job titles and permission tiers: the staff tier is labelled "Executive Chef", and a line cook must be made one just to log waste.	Goal-based navigation; permission-based role names with plain explanations in the invite window.
UX-A10–A14	MEDIUM	Forbidden pages silently redirect to the dashboard; platform admins have no link to the admin panel; in read-only (402) mode buttons look usable and fail afterwards, with no trial-ending warning; errors are vanishing toasts; every page is titled "Nexus"; new organizations get no onboarding.	Access-denied page; admin link; read-only styling and trial countdown; persistent errors; page titles; a getting-started checklist.

Area	Score /5	Area	Score /5
Visual design and consistency	4	Inventory and counts	2
Sign-in, sign-up, invitations	3.5	Waste logging	2.5
Purchasing and drafts	3.5	Sales (incl. #10)	2
Dashboard	3	Location switcher / All locations	2
Accessibility	3	App shell and navigation	2.5
Billing and read-only state	3	Onboarding / first run	1.5
Settings	3	**Mobile readiness (SOW)**	**2**

UX maturity by area, as scored by the app-wide reviewer (1 = poor, 5 = excellent).

9.5 · UX roadmap

Horizon	Changes
Quick wins (< 1 day each)	Discard-changes guard; unit-kind-change guard in review; "Unpriced" badges and cost caveats; "Open recipe" after confirm, with failures shown in the banner; Documents defaults to an allowed type; New Ingredient pre-filled with no kg default; searchable sub-recipe and menu pickers; units grouped by kind; persistent inline errors; accessible labels and ≥ 24 px targets
Medium (2–5 days each)	Inline unit bridge in the recipe editor (W1); review line converter plus cost column; new-ingredients preview (W3); one shared search picker; staff conversion proposals; phone card layouts; upload progress tray
Larger (1–3 weeks)	Full-page review workspace with source viewer and evidence highlighting (W2; pairs with §5 grounding); one recipe editor page (not a pop-up) with ingredients and sub-recipes together and an autosaved draft; organization custom units (your #9)

10 · Before Milestone 4: what to ask for

Viktor's Oct 12 summary frames the final milestone as "bug fixes and predictive ordering". The items below are corrections to Sprint 3 scope, or defects in work already delivered. They should be **Sprint 3 revisions**, not new Sprint 4 scope. Ordered by impact.

#	Must be done before Sprint 4 work begins	See
1	**SX-0 tenant-context leak**: per-request scope plus a regression test; then real database RLS. Ask the team to check staging for stray owner accounts created this way.	§8
2	**Invoice money safety**: CS→case and an invoice unit table (DOM-6/DX-3); missing unit is an error (DM-4); qty × price and invoice-total checks (DM-5); price-jump error at ~2×; "Confirm all" only for opened parts (DM-6); no SKU-over-name binding (DM-3).	§6, §7
3	**Extraction Phase 1** (§5.7): pinned model, no temperature, explicit thinking/effort, provenance; nullable prices; 'not this type'; Word text hygiene; keep "for frying"; truncation checks; retire pair mode; file-fingerprint reuse; photos inside PDFs at full resolution; one call per workbook sheet.	§5
4	**Your unit dead end** (#2/#3): inline "how many X is one Y" dialog; base-unit relabel when price and stock are zero; conversions don't lock units; no silent "each"; density rules must name an ingredient; Settings must not default to "Every ingredient".	§7
5	**Matching visibility**: a "matched to" column with match type; accept weak matches explicitly; stop stripping product-changing words (DM-1/DM-2).	§6
6	**Variance report** built on purchases and physical counts (DOM-2).	§7
7	**The benchmark the PM committed to on Sep 1**: 30–50 of your documents, precision and hallucination rate, 5 runs each, with the acceptance bar in §5.8. The §5.9 pilot shows the format.	§5.8–5.9
8	**Your workflow asks**: redirect to Documents after upload (#8); unsaved-changes guard (#7); default upload type under All locations (#1); sales parser never guesses columns (DM-7).	§2, §9

Strongly recommended alongside

- Extraction Phases 2–3: evidence per line, a grounding check, a source viewer in review, OCR for scans, a second read for invoices, sub-recipe structure built in code (§5.7).
- Duplicate-file warnings and an idempotent "apply newly mapped lines" (DM-8, DM-9); an organization date-order setting (DM-13).
- Security mediums: platform-admin separation (SX-1), decompression guard (SX-2), audit trail (SX-3).
- A decision from you on DOM-3: should invoice prices update recipe costs in multi-location groups, and how?
- Ask for incremental commits or PRs per fix instead of one 65,000-line squash; it makes revision review practical.

Appendix A · The production prompts today, verbatim

From `backend/src/modules/documents/extraction.ts` (lines 849–891 and 1007–1015). No provider uses a system instruction. For Gemini the prompt is placed *before* the file, against Google's guidance; for Claude it comes after.

Recipe

Extract this recipe as JSON: its name, how many portions it makes, the yield quantity and unit if stated, and one entry per ingredient with the ingredient name exactly as written, the quantity as a number and the measurement unit. When the quantity is a range or a choice (2 or 4, 2-3, 1 to 2), give its first figure as the quantity and copy the quantity exactly as written into quantityText; otherwise leave quantityText out. A line the paper gives no amount keeps its name, with no quantity and the phrase as written (to taste, to fry, for garnish) in note, or no note when there is none; never drop such a line and never write 0 for it. A component the file writes out with an ingredient list of its own (a sauce, dressing, dough or garnish) belongs to the recipe that uses it: return it under the subRecipes of that recipe, never as a separate entry in "documents", and keep a line for it among the items of that recipe, named as the component is named, with the quantity the recipe uses.

Invoice

Extract this vendor invoice as JSON. Keep each line item's raw ingredient name exactly as written. Quantities, units and unit prices are numbers; unit is the purchase unit (kg, l, case, each). Give sku only when the line prints an item code of its own, and lineTotal only when the line states its own total; copy both as written rather than recomputing or inventing them. Subtotal, tax and total rows are not line items and produce no line. Give invoiceDate as YYYY-MM-DD.

Sales report

Extract this sales report as JSON: the date of the sales it reports as YYYY-MM-DD, and one line per menu item sold, with the item name exactly as written, the quantity sold and the unit selling price as numbers. Lines that are not a menu item (category or department totals, taxes, discounts, tips, payments, deposits) produce no line.

Menu

Extract this menu as JSON: the name the menu prints for itself, if any, and one entry per dish with the dish name exactly as written, its selling price as a number, and the menu section it is listed under if the menu has sections.

Suffixes added to every prompt

[every prompt] + ' A single file may hold more than one; return each as a separate entry in "documents".'
[spreadsheet text] + ' The text comes from a workbook, one sheet after another under "Sheet:" headings. Sheets that hold no (a table of contents, stock counts, order guides, schedules) produce no entry.'

Pair-mode review prompt

You review a draft reading of a restaurant document. This is how the document is to be read:

The DOCUMENT is the source of truth. Find what the DRAFT got wrong: entries it missed entirely, entries that are not (a stray letter, a heading or a station name is not one), and lines that are missing, extra, or carry a wrong name, quantity, unit or price. Copy values as the paper writes them; do not invent.

Answer with corrections only: "add" holds whole entries the draft missed, each with every line the paper lists for it; "remove" the ids of draft entries that should not be there; "replace" an id with the whole corrected entry. Do not replace an entry only to restate a number with more or fewer digits. A line that names a sub-recipe keeps the sub-recipe's name exactly, so the two stay joined. Leave all three empty when the draft is right.

DRAFT:

Where these prompts invite errors: "how many portions it makes" has no "if stated"; "with the quantity the recipe uses" asks for an amount that is usually not printed; the invoice prompt gives "kg, l" as unit examples, which invites the model to convert lb→kg; "unit prices are numbers" plus a *required* price field forces a number when only a total is printed; the pair prompt asks the checker to add missing lines with no proof requirement; nothing allows "this is not a recipe" or "no ingredient list".

Appendix B · Proposed rewrites (evidence-first, tested Oct 12)

These implement the decision tree's "extract" step. They turn the model into a transcriber: it copies printed text with proof, and code does every conversion, calculation and check. Put the text in the system instruction (Gemini `systemInstruction` / Claude `system`) and send the file before the short user line. **This is the version measured in §5.9**, on Gemini 3.8 Flash, Gemini 3.5 Flash-Lite, Claude Haiku 5.5 and Claude Sonnet 5.5. The live test changed three things from the first draft: the one-ingredient-per-entry rule (rule 2), the narrower wrong-document rule (rule 9) and the workbook rule (rule 10). The prompt works together with code: **one call per workbook sheet**, **photos inside a PDF sent at full resolution** (plus the PDF's text layer), thinking set to low, and the checks of §5.5. Re-validate on the §5.8 benchmark before rollout.

B.1 Recipe extractor

```
You transcribe restaurant recipe documents into JSON. You are a transcriber, not a cook:
never add, complete, correct, convert or standardise anything.

1. ONLY WHAT IS PRINTED. Return a line only for a line printed in an ingredient list of the
   document. Do not add ingredients that appear only in the method or instructions, in a photo,
   in a note, or that recipes like this usually contain. If the document shows no ingredient
   list, return an empty "lines" list. An empty answer is correct; an invented line is a
   serious error.
2. EVIDENCE. For every line, copy into "evidence" the exact characters of that printed line,
   including its amount, unit and any notes, and give "page" (1-based; null for a file without
   pages). Do not fix spelling and do not join two printed lines.
   ONE INGREDIENT PER ENTRY. When one printed line or sentence lists several ingredients
   (separated by commas, "and", "+" or a colon after a component name, for example
   "Sauce: 1 cup soy, 2 tbsp honey, 1 tsp salt"), return one entry for each ingredient, each
   with its own name, amount and unit, and the whole printed line as the evidence of each.
3. VERBATIM FIELDS, NOT NUMBERS.
   - "nameText": the ingredient words exactly as printed ("shallots, minced"), without the
     amount, the unit or any price or cost printed beside it.
   - "quantityText": the amount exactly as printed ("1 1/2", "1/2", "2-3", "200"), or null.
   - "unitText": the unit exactly as printed ("c", "T", "tbsp", "g", "#10 can", "ea"), or null.
   - "amountNote": a printed phrase used instead of an amount ("to taste", "for frying",
     "as needed", "for garnish"), or null.
   Never turn fractions into decimals, never convert units, never compute.
4. NULL, NEVER A GUESS. When something is not printed, return null. Never return 0, 1 or any
   default amount, unit, yield or portion count.
5. SECTIONS AND COMPONENTS. Give each line the heading it is printed under, exactly as printed,
   in "section" (null if none). In "components", list every heading that has its own
   ingredient list: "nameText" as printed, "yieldText" as printed or null, and "usedAsText":
   the exact evidence of the line in another list that uses this component, or null if no
   printed line uses it. Never move a line to another section, never merge or split sections,
   never invent a line for a component.
6. TITLE AND YIELD. "titleText" is the recipe title as printed. "yieldText" and "portionsText"
   are copied exactly as printed, or null. Do not estimate either.
7. SEVERAL RECIPES. Return one entry in "recipes" per separately titled recipe with its own
   ingredient list, with the "pages" it spans. A component used by another recipe in the same
   file is not a separate recipe.
8. LEGIBILITY. If any character of a line is hard to read, still copy your best reading, and
   set "legibility" to "unclear" for that line; otherwise "clear".
9. WRONG DOCUMENT. Set "documentKind" to what the file mainly is. Return an empty "recipes"
   list only when the file contains no ingredient list at all (for example an invoice, a
   schedule, or a menu that only names dishes and prices). A menu, event sheet, prep sheet or
   book chapter that also gives ingredient lists is read as recipes: return every one of them.
10. WORKBOOKS. Text that comes from a workbook arrives one sheet after another under "Sheet:"
   headings. Sheets that hold no recipe (order guides, price lists, tables of contents, prep or
   par sheets, schedules) produce no entry, but every recipe on the other sheets is returned,
   and "documentKind" is "recipe" when any sheet holds one. In a table, take the amount and
   unit from the amount column only; cost and price columns are never amounts.

User turn, AFTER the file: "Transcribe the recipe document above following the rules."
Schema: every key required; absent values nullable (so the model must commit to null);
property order puts evidence first and parsed values last.
```

B.2 Invoice extractor

```
You transcribe vendor invoices into JSON. You are a transcriber, not an accountant: copy what is
printed; never calculate, convert, normalise, reorder or fill in.

1. ENTRIES. Return one entry per invoice (vendor + invoice number). A continuation page of the
   same invoice belongs to the same entry; list every page it spans in "pages".
2. HEADER, VERBATIM. "vendorText", "invoiceNumberText", "invoiceDateText" exactly as printed
   (for example "03/04/26"), or null.
3. ROWS. Return every printed row of the item table, in printed order, including fees,
   surcharges, deposits, credits, returns and discounts. For each row:
   - "evidence": the whole printed row exactly as on the page; "page".
   - "kind": item | fee | deposit | credit | discount | other.
   - "itemCodeText", "descriptionText" exactly as printed (description null only if none).
   - "qtyOrderedText", "qtyShippedText": as printed, or null.
   - "unitText": the unit-of-measure column as printed ("CS", "EA", "LB", "BX"), or null.
   - "packText": the pack/size as printed ("6/#10", "4/5 LB", "12/1 QT", "25 LB"), or null.
   - "unitPriceText": the per-unit price as printed, or null if the row prints none.
   - "priceUnitText": what the price is per, if printed ("/LB", "per CS"), or null.
   - "weightText": a printed catch weight ("41.6 LB"), or null.
   - "extendedPriceText": the row total as printed, or null.
   - "legibility": "unclear" if any character of the row is hard to read, else "clear".
4. NEVER COMPUTE. If a row prints only a total, unitPriceText is null. Never move a number to
   another column. Keep minus signs, brackets and "CR" exactly as printed.
5. TOTALS, VERBATIM. "subtotalText", "taxText", "otherChargesText", "totalText", or null.
   Subtotal, tax and total rows are not item rows.
6. WRONG DOCUMENT. Set "documentKind" to what the file is; if it is not an invoice, return no
   invoices.

Code afterwards (not the model): parse every number; quantity = shipped ?? ordered; parse the
unit with an INVOICE unit table (CS = case); a "/LB" price with a printed weight becomes weight x
price per lb; compute a missing unit price from the extended price and FLAG it as computed;
fees, deposits and credits never become stock lines; reconcile with the printed subtotal; check
every row's evidence against the document text.
```

B.3 If the full rewrite is deferred: minimum prompt edits

```
Recipe:  "how many portions it makes"  ->  "how many portions it makes if the document states it"
         delete "with the quantity the recipe uses"  ->  "with the quantity printed on that line, or
         no quantity if none is printed"
         append: "Return only lines printed in an ingredient list; never add an ingredient that is only
         mentioned in the method or that the document does not list; if there is no ingredient list
         return no items. When no amount is printed, leave the unit out too."
Invoice: "unit is the purchase unit (kg, l, case, each)"  ->  "unit is copied exactly as printed in
         the unit-of-measure column (for example CS, EA, LB)"
         add: "If a row prints no unit price, give its printed total in lineTotal and set unitPrice to
         null."  (schema: unitPrice nullable)
All:     add: "If the file is not a <type>, return an empty documents list."
Gemini:  remove temperature: 0; add thinkingConfig (low); send the file part first; pin the model
         version. Never change the model name without also setting the thinking level (§5.9).
Code:    workbooks: one call per sheet; PDFs that carry photos: send the photos at full resolution;
         invoices: CS = case, nullable unit price, fees and credits kept out of stock lines.
```